



UNIVERSIDAD NACIONAL DE CATAMARCA
FACULTAD DE TECNOLOGÍA Y CIENCIAS APLICADAS
TECNICATURA UNIVERSITARIA EN DISEÑO DE SOFTWARE

Diseño de Software y Patrones

TRABAJO PRÁCTICO N°1

Unidad N°3

DOCENTES:

- Carlos Acosta Parra
- Nancy Lamas

Alumna:

- Toloza Guadalupe del Valle MU: 00459

JUSTIFICACIONES

Ejercicio-B2

Propuesta 1:

Veredicto: Se rechaza.

Señal que falta: No existe variabilidad de clases ni lógica compleja de instanciación. Turno tiene una única implementación concreta con atributos directos.

Capacidad que NO se gana: No ganamos polimorfismo ni flexibilidad, ya que no hay otras clases concretas de turno que intercambiar.

Costo que se paga: Agregamos sobreingeniería. Introducimos una indirección innecesaria (una clase más que mantener y leer) sin resolver un problema real de diseño.

Solución: instanciación directa.

Propuesta 2:

Veredicto: Se acepta.

Señal concreta: Existe una restricción de coherencia. Tenemos una familia de objetos relacionados (redactor y emisor) que deben ser consistentes entre sí según el canal elegido.

Capacidad que se gana: Garantizamos por construcción que no exista una combinación inválida (por ejemplo, intentar usar un redactor de e-mail con un emisor de SMS).

Costo que se paga: Aumenta la estructura del sistema con una interfaz para la fábrica abstracta, una implementación concreta por canal y las interfaces de los productos.

Propuesta 3:

Veredicto: Se rechaza la implementación mediante Singleton (se resuelve con inyección de dependencias).

Justificación de las dos decisiones: El Singleton fuerza el acceso global para resolver la unicidad de la instancia. El acceso global genera acoplamiento oculto y dificulta las pruebas unitarias (por ejemplo, probar comportamiento en fechas específicas).

Costo: Se pierde flexibilidad para testing y modularidad al ocultar dependencias.

Solución correcta: Tratar a CalendarioDeFeriados como una clase común y corriente. Se crea la instancia una sola vez en la raíz de composición y se inyecta por constructor a las clases que la necesiten.

Propuesta 4:

Veredicto: Se rechaza.

Señal que falta: Falta la combinación de muchos parámetros, múltiples campos opcionales e invariantes que crucen varios campos.

Capacidad que NO se gana: No aporta ninguna flexibilidad de armado paso a paso ni manejo de opcionales, ya que todos sus campos son requeridos desde el inicio.

Costo que se paga: Se agrega código verboso y una clase constructora totalmente innecesaria para un objeto de dominio simple.

Solución directa: Instanciación directa a través del constructor o un objeto de parámetros sencillo.

Ejercicio-B3

Si las cinco herramientas se construyeran exactamente igual y ninguna necesitara datos propios, ¿seguiría justificándose la jerarquía de creadores?

Si las cinco herramientas se construyeran exactamente igual y ninguna necesitara datos propios, no sería necesario mantener una jerarquía de creadores con una subclase para cada herramienta. En ese caso utilizaría una única clase Herramienta parametrizada, indicando mediante un atributo qué tipo de figura debe crear.

Con la solución actual se tienen 6 clases relacionadas con las herramientas: una clase abstracta Herramienta y cinco clases concretas (HerramientaRectangulo, HerramientaElipse, HerramientaLinea, HerramientaCuadradoDeTexto y HerramientaFlechaAcotada). Con la alternativa se podría reducir a una sola clase Herramienta, ya que no existirían diferencias de comportamiento o datos propios que justifiquen las subclases.

Ejercicio-B9

¿Por qué el enunciado insiste en que las plantillas las arma el diseñador desde un editor? Si las unidades estuvieran definidas en el código, ¿qué solución más simple alcanzaría, y qué se perdería al usarla?

Si las unidades ya estuvieran definidas en el código y siempre fueran iguales, sería más sencillo hacer directamente `new Unidad(...)`. Pero en ese caso se perdería la ventaja de Prototype, porque cada vez que quisiéramos una unidad nueva tendríamos que definir nuevamente sus características en el código, en lugar de partir de una plantilla creada por el diseñador.

Ejercicio-B12

Decisiones de diseño

1. Tipos de aula — Sin patrón específico

¿Qué varía?

Varía el tipo de aula: común, laboratorio o aula magna, junto con sus características y reglas particulares.

¿Qué se gana y qué se paga?

Se separan las características de cada tipo mediante `Aula`, `AulaComun`, `LaboratorioInformatica` y `AulaMagna`. El costo es tener una clase por cada tipo.

¿Qué alternativa se descartó?

Se descartó una única clase `Aula` con condicionales para distinguir todos los tipos, porque concentraría reglas diferentes en una misma clase.

2. Notificaciones — Abstract Factory

¿Qué varía?

Varía el canal de comunicación: correo institucional o mensajería interna. Cada canal tiene su propio redactor y emisor.

¿Qué se gana y qué se paga?

`FabricaMail` y `FabricaMensajeria` permiten crear familias de productos compatibles y cambiar de canal sin modificar `ServicioNotificacion`. El costo es incorporar fábricas y productos concretos.

¿Qué alternativa se descartó?

Se descartó crear directamente los redactores y emisores dentro de `ServicioNotificacion`, porque aumentaría su acoplamiento con cada canal.

3. Construcción de reservas — Builder

¿Qué varía?

Varían principalmente los datos opcionales de la reserva y las combinaciones de datos que deben validarse.

¿Qué se gana y qué se paga?

`ReservaBuilder` permite construir la reserva progresivamente y validar las reglas antes de crearla. El costo es agregar el builder y mantener allí estas validaciones.

¿Qué alternativa se descartó?

Se descartó un constructor con todos los parámetros y validar después, porque permitiría crear reservas inválidas.

4. Configuración y confirmación

¿Qué varía?

La configuración puede cambiar independientemente del resto del sistema. Al confirmar una reserva deben coordinarse notificaciones y registro en el calendario.

¿Qué se gana y qué se paga?

ConfiguracionSistema concentra la configuración. ServicioReserva coordina la confirmación y recibe Notificador y Calendario como dependencias. CalendarioInstitucional registra la reserva. Así Reserva no conoce servicios externos y las dependencias pueden sustituirse en las pruebas. El costo es incorporar estas clases y abstraer sus dependencias.

¿Qué alternativa se descartó?

Se descartó que Reserva realizara las notificaciones y el registro directamente, porque mezclaría responsabilidades del dominio con servicios externos.

5. Duplicación de reservas — Prototype

¿Qué varía?

Al duplicar una reserva se mantienen sus datos y solamente cambia la fecha. Además, se crea un nuevo array para las observaciones, copiando sus elementos.

¿Qué se gana y qué se paga?

El patrón permite duplicar una reserva sin reconstruirla desde cero y evita que la copia y la original compartan el mismo array de observaciones. El costo es implementar la lógica de clonación.

¿Qué alternativa se descartó?

Se descartó crear manualmente una nueva reserva copiando todos sus datos, porque sería más repetitivo y podría producir una copia incompleta.